

Understanding AtScale OLAP Cubes and Their Data Model

AtScale provides a *virtual* OLAP cube layer on top of data warehouses (like BigQuery), allowing data to be modeled as **dimensions** and **measures** in a cube format without physically pre-aggregating all data ¹. In an OLAP cube, **measures** are numeric metrics (e.g. Sales, Units) that are aggregated, and **dimensions** are categorical fields (e.g. Product, Date, Region) used to slice and filter those measures ². AtScale's cube metadata (often represented in JSON or XML form) captures all the definitions of these dimensions, hierarchies, measures, and calculated members, which you can use to drive a search or query-building service. ³ ⁴

Overview of AtScale Virtual Cubes and OLAP Structure

Conceptual view of an OLAP cube: measures (e.g. Sales) aggregated by various dimensions (e.g. Product, Time, Region). Each "cell" of the cube is a value (like total Sales) at a specific intersection of dimension values.

An **OLAP cube** organizes data in a multi-dimensional structure, where each dimension provides a different context for analysis (e.g. time, product, geography). **AtScale cubes** implement this concept virtually: they overlay a multidimensional schema on the underlying relational data without copying or pre-summarizing all combinations ⁵ ¹. This means the data stays in the data warehouse tables, but AtScale's engine knows how to interpret queries using the cube's metadata, generating optimized SQL on the fly. AtScale uses a **hybrid MOLAP/ROLAP** approach – it appears as a traditional cube to BI tools (with friendly lists of measures and dimensions), but under the hood it performs relational queries and can even build **aggregate summary tables** dynamically based on query patterns to improve performance ¹. In other words, AtScale's query optimizer may pre-create certain aggregated results ("aggregate indexes") as queries run, combining the scalability of relational databases with the speed of precomputed cubes.

From a data modeling perspective, an AtScale cube is usually based on a central **fact** dataset/table (containing the measurable events, e.g. sales transactions) surrounded by **dimension** datasets (containing categorical context like dates, products, customers) ⁶ ⁷. This mirrors the classic **star schema** design: one fact table linked to multiple dimension tables ⁶. However, AtScale's *virtual* modeling is flexible – a dimension in AtScale doesn't have to be a single physical table; it could be defined directly on a fact table column or even a logical combination of columns from multiple tables ⁷. (For example, a *degenerate dimension* might be created from a fact table's own column, like an order number or transaction type, treated as a dimension for filtering.) The key is that each dimension must be **related** to the fact data via defined relationships (join keys): "A dimension is not considered part of a cube until it has a relationship to the fact data (either directly or indirectly)" ⁸. These relationships tell AtScale how to join dimension data to fact data at query time (typically linking a dimension's key attribute to a foreign key in the fact) ⁹. If multiple fact tables (measure groups) exist in one cube, each dimension would be related to one or more facts; AtScale's engine uses the relationship definitions to determine valid join paths and query behavior for any given combination of measures and dimensions ¹⁰. (AtScale even has settings for handling **unrelated**

dimensions – cases where a requested dimension has no direct relationship to a chosen measure – to decide whether to show all values aggregated or treat it as no data ¹¹ .)

In summary, an AtScale cube presents a **semantic layer** where all possible measures and dimensional breakdowns are defined in metadata. This metadata is what you'll leverage for building your search/query tool – it's essentially a complete map of the analytical entities (metrics, categories, hierarchies) available to end-users.

Dimensions, Hierarchies, and Attributes in a Cube

Dimensions in a cube are the categorical fields by which you can group or filter measures – they answer questions like “*who?*”, “*what?*”, “*where?*”, “*when?*” for your data ¹² . In the cube's JSON definition, each dimension will typically have a unique ID and a set of **properties** (such as name, descriptions, etc.), and importantly one or more **hierarchies**. A *hierarchy* organizes dimension **attributes** into levels of granularity ¹³ . For example, a Date dimension might have a hierarchy called “Calendar” with levels for Year > Quarter > Month > Day. Each level is a set of attributes representing that grain (Year Name, Quarter Name, etc.), and the lowest level of a hierarchy is the **key level** (the most granular attribute, e.g. Date or GeoKey) which uniquely identifies a dimension member ¹⁴ . The key level attribute is usually what links to the fact table (e.g. Date ID, Product ID) for joining ¹⁵ .

Under the hood, a dimension's hierarchy and levels map to one or more underlying table columns. AtScale's model ensures each hierarchy level evenly rolls up to the next (each Month belongs to one Quarter, etc.), enforcing proper aggregation behavior ¹⁶ . In the JSON, you might see each level with an ID, name, and possibly a reference to the source column. For instance, a “Product” dimension could have a hierarchy “Product Categories” with levels: Category, Subcategory, Product Name, each level corresponding to a column or expression. The cube metadata might list something like:

```
"dimensions": [  
  {  
    "name": "Product",  
    "id": "Dim-Product",  
    "hierarchies": [{  
      "name": "Product Categories",  
      "levels": [  
        {"name": "Category", "id": "Level-Category", "keyColumn":  
"product_category_id", [...]},  
        {"name": "Subcategory", "id": "Level-Subcategory", [...]},  
        {"name": "Product", "id": "Level-Product", "keyColumn":  
"product_id", [...]}  
      ]  
    }],  
    [...] (other properties)  
  },  
  [...] (other dimensions)  
]
```

Each **dimension attribute** can be one of two types in AtScale's design: a **level attribute** (part of a hierarchy as above) or a **secondary attribute** ¹⁷. *Level attributes* are tied to a specific hierarchy level and are used for the primary drill-down paths and for relating to fact data ¹⁸. *Secondary attributes* are additional attributes about the dimension that aren't on the main hierarchy path ¹⁷. For example, a Customer dimension's hierarchy might be geographic (Country > State > City), but the dimension could have secondary attributes like *Gender* or *Birth Date* that don't fit into that hierarchy tree ¹⁷. These attributes still belong to the dimension and can be used to filter or display, but they aren't used as levels of aggregation (and typically wouldn't appear in a drill-down hierarchy). In the JSON, secondary attributes might be listed separately or flagged differently (they could appear as additional properties of a level or as a separate list of attributes). It's important to capture these too for search purposes, since users might query on them (e.g. "female customers" involves the Gender attribute of Customer dimension).

Every dimension in the cube is linked to the fact dataset via a relationship on a key. This information might not be explicit in the high-level JSON (which focuses on logical structure), but it's defined in the project's model. When building your search tool or data graph, you can assume that any dimension listed in the cube's metadata is *meant* to slice the cube's measures. (If the cube JSON includes a section for **relationships**, it would specify which dimension level corresponds to which fact column as the foreign key ⁹.) For a single-fact cube, all dimensions are generally applicable to all measures. If the cube encompasses multiple fact tables (measure groups), the metadata's relationship definitions will tell you which dimensions apply to which fact. In practice, AtScale's "Cube Matrix" or relationship view would highlight if a particular measure is unrelated to a certain dimension. You might incorporate that logic by constructing a mapping of measure → related dimensions (and vice versa) from the metadata, to validate query combinations (this is akin to building a "truth table" of which dimensions are valid for each measure).

Measures and Calculated Members in a Cube

Measures are the quantitative metrics in the cube – typically aggregations like sum of sales, count of transactions, average unit price, etc. ³ ¹⁹. In your cube's JSON, measures might be listed under a section often called "*measures*" or possibly "*attributes*" (depending on terminology). Each measure entry usually has properties such as a name, an ID, the underlying column it's derived from, and the aggregation type (Sum, Count, Avg, etc.) ¹⁹. For example, a measure definition in JSON might look like:

```
{
  "name": "Sales",
  "id": "Measure-Sales",
  "sourceColumn": "sales_amount",
  "aggregation": "SUM",
  "folder": "Financial Metrics",
  "description": "Total sales in dollars",
  "dataType": "FLOAT"
}
```

Here, **Sales** is defined as a sum of the `sales_amount` column from the fact table. Measures can have different aggregate types (count distinct, average, min, max, etc.), and AtScale even supports semi-additive measures (where certain dimensions aggregate differently, like inventory which might take a *last value* over time) ²⁰ ²¹. Each measure typically has a *user-friendly display name* and possibly a separate *query name* or

alias that the engine uses in generated queries ²². (AtScale allows specifying a “Query Name” which might default to a safe SQL alias, and a description to help users understand the measure ²³.) Measures are **always tied to the fact dataset** in AtScale – you can’t create a measure on a pure dimension table. (There is an experimental concept of “measure within a dimension” for specialized cases, but generally, measures come from the fact data ²⁴.)

To help organize measures (especially when there are many), cubes often use **display folders**. A folder is just a logical grouping – for example, measures might be grouped into “Sales Metrics”, “Inventory Metrics”, “Ratios” etc. In the JSON, each measure may have a property like `"folder": "Sales Metrics"`. This causes BI client tools to show those measures under a common node. *“If your cube has a lot of measures, folders are a good way to organize them.”* ²⁵. The example above shows *Sales* in the *Financial Metrics* folder. Your cube JSON likely has similar folder assignments for measures (and possibly for calculated measures too, so they can appear in a grouped manner).

Beyond base measures, cubes often define **calculated members** (also called calculated measures when they result in a new measure). These do not directly come from a single column, but are defined by a formula – for instance, *Avg Unit Retail* might be a calculated measure defined as `Sales / Units` (total sales divided by total units sold). In AtScale (and OLAP in general), calculated measure formulas are usually written in **MDX (Multi-Dimensional Expressions)** syntax ²⁶. The cube metadata will include these calculated members with an expression string. For example, the JSON might have an entry:

```
{
  "name": "Avg Unit Retail",
  "id": "Calc-AvgUnitRetail",
  "expression": "([Measures].[Sales] / [Measures].[Sales Units])",
  "folder": "Financial Metrics",
  "formatString": "Currency",
  "description": "Average price per unit sold"
}
```

The `expression` here is an MDX formula referencing other measures in the cube. According to AtScale documentation, *“Calculated measure formulas are written in MDX syntax, but only support a subset of MDX functions and operators. They can only operate on existing measures in the cube, and must return a numeric value.”* ²⁶. This means you’ll see formulas that combine measures with arithmetic or simple conditional logic (e.g., dividing one measure by another, or using an `IIF` for conditional results). AtScale provides an MDX function reference and even suggests measure names as you write the formula, indicating it expects formulas like `[Measures].[X] + [Measures].[Y]` etc. ²⁷. In the JSON, the calculated measure might also have a `folder` property (as shown) and a format or other metadata.

When parsing these, note that the *name* might be what users see, whereas the *ID* or the *query name* would be used internally or in MDX queries. For instance, AtScale often has a concept of a *display name* vs. *query name* for measures ²², and the MDX expression likely uses the query names. If your search tool needs to present these calculated measures, you may want to index both the name and perhaps keywords in the expression (like “average” in *Avg Unit Retail*), but typically users will search by the name or business term.

Cube Metadata Structure and Internal Considerations

Bringing it all together, the cube's metadata (the "huge JSON" you mentioned) likely contains sections for **dimensions**, **measures/attributes**, and **calculations**, each with numerous properties. In an AtScale project JSON, these objects are identified by unique IDs and have references to each other (for example, a relationship might refer to a dimension's key level ID and a dataset column ID). Some key structural points and internal considerations to be aware of:

- **Dimension definitions:** Each dimension entry will include its hierarchies and levels (with keys, names, possibly captions, data types, etc.). The **key level** is crucial as it links to fact data ¹⁴. The hierarchy structure can be leveraged to understand parent-child relationships (e.g. if a search query mentions "2021 Q1", your tool might need to recognize this could relate to *Year 2021 > Quarter 1* in the Date dimension hierarchy). You might consider indexing level names and perhaps some representative **members** if available (though the cube metadata typically doesn't enumerate all members; you'd retrieve those via queries). If you have **precomputed indices of dimension values** (e.g. a list of all product names, all date values, etc.), those would be used at query time to suggest filter values. Keeping those separate from the static cube structure is wise, as dimension members can be very large lists.
- **Measure and attribute list:** The JSON's "attributes" section that you mentioned likely contains the measures (and possibly any non-hierarchical attributes, if AtScale uses that term). Confirm by the names – if you see things like Sales, Sales Units, etc., those are measures. Each will have metadata like aggregation type (Sum, Count, etc.), data type, and folder grouping ²⁵. Some might have `visible = false` if they're only used internally for calculations (AtScale allows measures to be hidden from users if only used in a calculated measure) ²⁸. Your search index should probably include all visible measures' names (and maybe descriptions) so users can find them by business terms.
- **Calculated members:** As discussed, these are separate entries with expressions. They might be under a "calculatedMeasures" or "calculations" section in the JSON. Internally, AtScale treats these like additional measures computed at runtime. They will not correspond to a physical column but to an MDX formula on the cube. Be mindful that some calculated members could be for special purposes (like time intelligence – e.g. Year-to-date calculations might be present). AtScale also has a concept of **calculation groups** (for dimensions, e.g. a Time Calculation dimension that can apply a formula like YTD across multiple measures), but those would appear as special dimensions or calculations in metadata ²⁹ ³⁰. Unless your cube uses those, you can focus on the individual calc measures.
- **Relationships and valid combinations:** Internally, the cube knows which dimensions apply to the fact (and thus to measures). For a simple cube with one fact, **every dimension is related to every measure** by default. If your cube has multiple facts (like a "Sales" fact and a "Budget" fact combined in one cube), then some measures come from one fact and some from another. AtScale's model would then have each dimension either connected to both facts or only one. This is where the *cube matrix* (or a "truth table" of measures vs dimensions) comes in. AtScale ensures a query can still run even if you use an unrelated dimension with a measure (via its *Unrelated Dimensions Handling* setting) ¹¹, but the result might be not what the user expects (often it shows the total of the measure for all values of that unrelated dimension, repeated for each value). In a search or assistive

tool, you might prefer to guide users to valid combinations. **To do this, use the relationship info:** for each measure, know which dimensions it's connected to (directly or via shared dimensions). In your data structure or graph, you could create links between measure nodes and dimension nodes to represent "measure X can be analyzed by dimension Y". In a fully related star schema, this is all dimensions for all measures. In more complex cubes, it's a subset. The AtScale metadata *implicitly* provides this via the relationships definitions (dimension to fact links). You may need to derive a mapping like `measure -> fact dataset -> related dimensions` (since each measure knows which fact/dataset it comes from, and each dimension relationship tells which fact(s) it connects to). Once derived, that mapping is your *truth table* to validate query context.

- **Performance considerations:** Under the hood, when a query is run (say a user drags a measure into a BI tool with some dimension filter), AtScale's engine will translate that into SQL on the warehouse. It may leverage pre-built **aggregates** (AtScale automatically can create aggregate tables in the data warehouse as a cache of query results) ¹. This is mostly transparent but worth noting: if your search tool might allow very ad-hoc combinations, initial queries could hit raw data but subsequent ones might hit aggregates. From a metadata perspective, you might not see those aggregates – they are not defined in the cube JSON; they are managed at runtime. What *is* defined is things like **aggregate awareness** (the measures that drive aggregate creation) ³¹ – essentially, every measure in the cube is a candidate for aggregation, and the presence of measures influences what aggregates get built. For your purposes (search and query-building), you mostly need to trust that any combination you allow that's valid in the cube will be handled by AtScale with as much optimization as it has available. You do not need to manually factor in the aggregates, except possibly being aware of any *special measures* (like distinct counts or non-additive measures) which might not aggregate nicely across all dimensions.

Leveraging Cube Metadata for a Search/Query Service

With an understanding of the cube's structure, you can now think about how to use it to power a search or query suggestion feature. Essentially, the cube metadata is a **graph of analytical concepts**: dimensions connected to measures through the fact. You can utilize this as follows:

- **Indexing key entities:** Catalog all dimension names, hierarchy level names, and measure names (including calculated ones). Also include synonyms or descriptions. For instance, if a measure description says "Total Revenue", a user search for "revenue" should match the *Sales* measure. Likewise, if a dimension level is "Product Category" and the user searches "category", you'd want that in results. Storing the *folder* grouping might help categorize results (e.g. show that Avg Unit Retail is a "Financial Metric" vs something else).
- **Including dimension values:** Likely you have separate pre-calculated indices for dimension **members** (values). For example, an index of all product names, all customer names, etc., possibly with IDs. Those are crucial for filter suggestions. Connect those to their dimension in your knowledge graph. For instance, the value "Acme Corp" would link to the Customer dimension's lowest level. That way, if a query mentions "Acme", you identify it as a Customer dimension member and can formulate a filter on Customer = Acme.
- **Graph of relationships:** Build a graph where **nodes** are measures and dimensions (and possibly dimension *levels* as distinct nodes), and **edges** connect measures to all the dimensions that are

related (i.e., the measure's fact has a relationship to that dimension). Also connect dimension levels to their parent dimension and to the measure edges for context. This graph can help you interpret multi-part queries: e.g. if user searches "sales in 2020 for electronics", your system sees "Sales" (measure), "2020" (likely a Year member in Date dimension), "Electronics" (maybe a Product Category member). Your graph would show that *Sales* is connected to *Date* and *Product* dimensions, and that "2020" is a member of Date dimension, "Electronics" a member of Product dimension – thus all three can form a valid query together. The graph also helps to exclude combinations that don't make sense (if a dimension wasn't linked to that measure, there'd be no edge, and you could flag that or handle it differently).

- **Cube internals to consider:** Pay attention to details like **role-playing dimensions** (the same logical dimension used multiple times, e.g. Order Date vs Ship Date as two roles of a Date dimension). In AtScale's metadata, those might appear as separate dimensions (often with a prefix in the query name to distinguish roles ³²). For search, you may want to treat them as separate (if the user says "Order Date", vs "Ship Date"). Also, if any **many-to-many dimensions** or **bridge tables** exist (AtScale supports multi-valued dimensions ³³ ³⁴), be aware that filtering on those might affect measures differently. However, these are advanced cases; the majority of cubes have straightforward one-to-many relationships.
- **Validation (Truth Tables):** As noted, implement a check so that when a user picks a measure and some dimension filters, you verify that the measure is aggregable by those dimensions. Since AtScale can technically return a result even if a dimension is unrelated (often treating it as if the measure isn't filtered by that dimension), you may want to either prevent those combinations or label them. For example, if *Budget* measure isn't linked to *Product Category* dimension (perhaps Budget is only by Month and Region), a query "Budget by Product Category" might yield each category showing the total Budget (which could be misleading). The cube metadata relationships would indicate no direct link between Budget's fact and the Product dimension. Your tool could catch that and warn the user or handle it by not drilling into that dim for that measure.

In essence, **AtScale's cube JSON provides a blueprint of the semantic layer**: all measures (with their definitions and groupings), all dimensions (with structures and attributes), and how they join. Learning this structure – which is similar to other OLAP systems (like Microsoft SSAS or Mondrian in concept) – lets you build a robust search index. You'll treat measure names, dimension names, and dimension values as the corpus of terms. The hierarchical relationships give you context (for example, knowing "New York" is a City under "USA > NY State" in the Geography dimension might let you broaden/narrow search results). And the measure-dimension relationships guide you in assembling valid query candidates from search terms.

Conclusion

By studying the AtScale cube's data model, you gain insight into **how the data is organized and can be queried**. A cube has well-defined pieces – *dimensions with hierarchies/levels*, *measures (base and calculated)*, and the *relationships* tying it all together – all of which are described in the cube's JSON schema. This mirrors standard OLAP structures, so even general OLAP knowledge (like what a dimension or hierarchy is) applies directly. AtScale's documentation confirms these concepts: "A dimension is a logical collection of attributes... used to group and filter measure data" ¹³; "A measure is a numeric value representing a summarized dataset metric (such as sum of sales)... Measures always result from an aggregate calculation on one or more columns of a fact dataset." ¹⁹. Calculated measures extend these by allowing MDX formulas on existing measures ²⁶.

With this understanding, you can parse that “huge JSON” more confidently — identifying all the metrics, dimensions, and allowed filter values — and then engineer a search service that intelligently maps user queries to the cube’s components. Not only will this help in retrieving the right data, but it will also educate your users about the available data (since the cube’s semantic layer is essentially a dictionary of the business data model). Armed with the cube’s metadata and some graph/index structures, you’ll be well on your way to building a powerful query suggestion and validation system on top of your AtScale semantic layer.

Sources:

- AtScale Documentation – *About Cube Dimensions* ¹³ ¹⁴ , *About Dimension Attributes* ¹⁷ , *About AtScale Virtual Cubes* ¹ ² , *About Measures and Dimensions* ³ ⁴ , *Modeling Cube Measures* ¹⁹ , *Add Additive Measures* ²⁵ ²³ , *Add Calculated Measures* ²⁶ ³⁵ , *About Star Schema Data Modeling* ⁷ , *Modeling Relationships* ⁸ ⁹ .
-

1 2 5 **About AtScale Virtual Cubes | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/atscale-cube-design-concepts/about-atscale-virtual-cubes>

3 4 12 **About Measures and Dimensions | AtScale Documentation**

<https://documentation.atscale.com/installer/next/creating-and-sharing-cubes/atscale-cube-design-concepts/about-measures-and-dimensions>

6 7 **About Star Schema Data Modeling | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/atscale-cube-design-concepts/about-star-schema-data-modeling>

8 9 10 **Modeling Relationships | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/creating-cubes/modeling-relationships>

11 19 24 31 **Modeling Cube Measures | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/creating-cubes/modeling-cube-measures>

13 14 15 **About Cube Dimensions | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/creating-cubes/modeling-cube-dimensions/about-cube-dimensions>

16 **About Dimension Hierarchies and Levels - AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/creating-cubes/modeling-cube-dimensions/about-cube-dimensions/about-dimension-hierarchies-and-levels>

17 18 **About Dimension Attributes | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/creating-cubes/modeling-cube-dimensions/about-cube-dimensions/about-dimension-attributes>

20 21 22 23 25 28 **Add Additive or Non-Additive Measures | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/creating-cubes/modeling-cube-measures/add-additive-or-non-additive-measures>

26 27 29 30 32 35 **Add Calculated Measures | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/creating-cubes/modeling-cube-measures/add-calculated-measures>

33 34 **Types of Dimensions in a Cube | AtScale Documentation**

<https://documentation.atscale.com/installer/creating-and-sharing-cubes/creating-cubes/modeling-cube-dimensions/about-cube-dimensions/types-of-dimensions-in-a-cube>